Lecture 8 — Geometry for Regionals

> Regional Level · Prerequisites: Year Curriculum Week 31 (vectors, cross product, convex hull) · Estimated time: 2.5 hours

1. Overview

Most regionals contain one or two geometry problems, and many teams skip them out of fear: special cases, collinear points, floating-point errors. In practice, regional geometry is usually medium difficulty *if* you have a small, tested core and follow one rule: **when the input is integral, keep every decision integral**. Floating point is used only to compute an output value, never to decide a branch.

This lecture builds that core:

- points, cross and dot products, orientation,
- exact segment intersection and line intersection points,
- polygon area, Pick's theorem, point in polygon (general and convex),
- convex hull by monotone chain,
- precision rules, angle sorting without `atan2`, and circle basics.

Rotating calipers, half-plane intersection, and closest pair are in the National track (Lecture `nationals-08`).

2. The core

```
struct P {
    ll x, y;
    P operator-(const P& o) const { return {x - o.x, y - o.y}; }
    bool operator<(const P& o) const { return x != o.x ? x < o.x : y < o.y; }
};
ll cross(P a, P b) { return a.x * b.y - a.y * b.x; }
ll cross(P o, P a, P b) { return cross(a - o, b - o); }
ll dot(P a, P b) { return a.x * b.x + a.y * b.y; }
```

`cross(o, a, b)` is twice the signed area of triangle `o, a, b`. It is positive when `b` lies to the left of the directed line `o → a` (a counter-clockwise turn), negative to the right, and zero when the points are collinear. `dot(a, b)` is positive when the angle between `a` and `b` is acute.

Magnitudes. With coordinates up to 10^9, differences reach $2 \cdot 10^9$ and a cross product reaches $8 \cdot 10^{18}$, which is just below the `long long` limit of about $9.2 \cdot 10^{18}$. Coordinates up to 10^9 are therefore safe for a single cross product but not for sums of them or anything larger; use `__int128` when in doubt.

3. Segments and lines

3.1 Segment intersection

Two segments `ab` and `cd` intersect iff either they cross properly (`c` and `d` on strictly opposite sides of line `ab`, and `a`, `b` on strictly opposite sides of line `cd`) or an endpoint of one lies on the other. The second case covers touching at an endpoint and collinear overlap. "Lies on" means collinear (cross product zero) and inside the bounding box of the segment. This is exact for integer coordinates and handles every special case, including degenerate segments, without any epsilon. It solves *CSES 2190 Line Segment Intersection*.

3.2 Line intersection point

Write the lines as `A + t·r` and `C + u·s` with `r = B − A`, `s = D − C`. Taking the cross product of `A + t·r = C + u·s` with `s` gives

```
t = cross(C − A, s) / cross(r, s).
```

If `cross(r, s) = 0`, the lines are parallel; if additionally `cross(C − A, r) = 0`, they are the same line. Decide these cases with integers; compute only the final point in `long double`:

```
ll den = cross(r, s);
if (den == 0) { /* parallel, or the same line if cross(C - A, r) == 0 */ }
long double t = (long double)cross(C - A, s) / den;
// point = (A.x + t * r.x, A.y + t * r.y)
```

The intersection of two segments, when it is a single point, is the same computation after the exact test of 3.1. Kattis `segmentintersection` asks for that point (or the overlap segment), with the usual output-formatting traps such as printing `-0.00`.

4. Polygons

4.1 Area

The shoelace formula: `2A = Σ cross(p_i, p_{i+1})` with indices modulo `n`. The sum is positive for counter-clockwise order, so its sign also tells you the orientation (Kattis `polygonarea` asks for both). With integer vertices `2A` is an integer; print the area as an exact `.0` or `.5` rather than through a double.

4.2 Pick's theorem

For a simple polygon with integer vertices, `2A = 2I + B − 2`, where `I` is the number of interior lattice points and `B` the number of boundary lattice points. Each edge from `p` to `q` contains `gcd(|q.x − p.x|, |q.y − p.y|)` lattice points not counting one endpoint, so `B` is the sum of these gcds. Then `I = (2A − B + 2)/2`. This is *CSES 2193 Polygon Lattice Points*.

4.3 Point in polygon

Ray casting: shoot a ray from `p` to the right and count how many edges it crosses; odd means inside. The difficulty is rays passing exactly through vertices or along horizontal edges. The **half-open rule** fixes it: an edge `a → b` (with `a.y ≤ b.y` after swapping) is counted only if `a.y ≤ p.y < b.y` and `p` is strictly to the left of the edge (`cross(b − a, p − a) > 0`). Horizontal edges are never counted, and a vertex is counted for exactly one of its two edges when the ray passes through it. Test the boundary first with the exact `onSegment` predicate. This is *CSES 2192 Point in Polygon*, O(n) per query.

4.4 Point in a convex polygon in O(log n)

For a convex polygon `v_0, …, v_{n−1}` in counter-clockwise order with no three collinear vertices, the diagonals from `v_0` split it into a fan of triangles. If `p` is to the right of `v_0 → v_1` or to the left of `v_0 → v_{n−1}`, it is outside. Otherwise, binary search the largest `k` with `cross(v_0, v_k, p) ≥ 0`; then `p` lies in the wedge between `v_k` and `v_{k+1}` and it remains to check the side of the outer edge `v_k → v_{k+1}`. Points on the first or last fan edge are on the boundary. *Codeforces 166B Polygons* (are all vertices of polygon B strictly inside convex polygon A, with up to 10^5 and $2 \cdot 10^4$ vertices?) needs exactly this; note that its polygons are given clockwise, so reverse them first.

5. Convex hull

Andrew's monotone chain: sort points by `(x, y)`, remove duplicates, build the lower hull from left to right while popping points that do not make a strict left turn, then the upper hull from right to left. The result is in counter-clockwise order starting at the lowest-leftmost point. O(n log n).

Before submitting a hull, settle these questions from the statement:

- **Duplicates**: remove them, or the popping condition misbehaves.
- **Collinear boundary points**: the condition `cross ≤ 0` drops them; `cross < 0` keeps them, but then the all-collinear case returns points twice and needs separate handling. Kattis `convexhull` wants them dropped; CSES 2195 wants them kept.
- **Tiny inputs**: one point, two points, all points collinear.
- **Output order**: clockwise or counter-clockwise, and which vertex first.

Hulls appear as sub-steps: farthest pair of points (only hull vertices matter), the minimum-length fence, the set of points visible from infinity, and the upper envelope behind the convex hull trick.

6. Precision

1. **Integer input means integer predicates.** Orientation, intersection tests, point-in-polygon, and distance comparisons (compare squared distances) are all exact in integers.
2. **Avoid `sqrt`, `atan2`, `acos`** in decisions. `acos` of a value slightly above 1 because of rounding returns NaN.
3. **If floating point is unavoidable**, use `long double` and an epsilon adapted to the scale of the values (`1e-9` is meaningless if coordinates are 10^9 and products are 10^{18}).
4. **Never compare doubles with `==`.**
5. **Sort by angle with cross products**, not `atan2`: split the plane into two half-planes and compare by cross product within a half:

```
int half(P p) { return p.y < 0 || (p.y == 0 && p.x < 0); }
bool angleLess(P a, P b) {
    if (half(a) != half(b)) return half(a) < half(b);
    return cross(a, b) > 0;
}
```

Codeforces 598C Nearest vectors (find the pair of vectors with the smallest non-oriented angle among 10^5 vectors with coordinates up to 10^4) is the famous example: sorting with `atan2` in `double` and comparing angle differences fails on adjacent nearly-equal angles; `long double`, or an exact comparison of the angle differences via dot and cross products, passes.

6.1 Circles

Point in circle: compare `dist² ≤ r²` exactly. Line–circle intersection: project the center onto the line, then use Pythagoras for the half-chord. Circumcenter of three points: solve the 2×2 system from equal distances. *Codeforces 1C Ancient Berland Circus* (smallest-area regular polygon with three given vertices) combines the circumcircle with a floating-point gcd of central angles, and is a good exercise in choosing tolerances.

7. Reference implementations

7.1 CSES 2190 — Line Segment Intersection

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct P {
    ll x, y;
    P operator-(const P& o) const { return {x - o.x, y - o.y}; }
};
ll cross(P a, P b) { return a.x * b.y - a.y * b.x; }
int orient(P a, P b, P c) {
    ll v = cross(b - a, c - a);
    return (v > 0) - (v < 0);
}
bool onSegment(P a, P b, P c) {          // c is known to be collinear with ab
    return min(a.x, b.x) <= c.x && c.x <= max(a.x, b.x) &&
           min(a.y, b.y) <= c.y && c.y <= max(a.y, b.y);
}
bool segmentsIntersect(P a, P b, P c, P d) {
    int o1 = orient(a, b, c), o2 = orient(a, b, d);
    int o3 = orient(c, d, a), o4 = orient(c, d, b);
    if (o1 * o2 < 0 && o3 * o4 < 0) return true;
    if (o1 == 0 && onSegment(a, b, c)) return true;
    if (o2 == 0 && onSegment(a, b, d)) return true;
    if (o3 == 0 && onSegment(c, d, a)) return true;
    if (o4 == 0 && onSegment(c, d, b)) return true;
    return false;
}

int main() {
    int t;
    cin >> t;
    while (t--) {
        P a, b, c, d;
        cin >> a.x >> a.y >> b.x >> b.y >> c.x >> c.y >> d.x >> d.y;
        cout << (segmentsIntersect(a, b, c, d) ? "YES" : "NO") << '\n';
    }
}
```

Verified against an exact rational-arithmetic brute force on thousands of random segment pairs with small coordinates (many collinear and touching cases).

7.2 CSES 2192 — Point in Polygon

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct P {
    ll x, y;
    P operator-(const P& o) const { return {x - o.x, y - o.y}; }
};
ll cross(P a, P b) { return a.x * b.y - a.y * b.x; }
bool onSegment(P a, P b, P p) {
    return cross(b - a, p - a) == 0 &&
           min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) &&
           min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}

int pointInPolygon(const vector<P>& poly, P p) {   // 0 out, 1 in, 2 boundary
    int n = poly.size();
    bool inside = false;
    for (int i = 0; i < n; i++) {
        P a = poly[i], b = poly[(i + 1) % n];
        if (onSegment(a, b, p)) return 2;
        if (a.y > b.y) swap(a, b);
        if (a.y <= p.y && p.y < b.y && cross(b - a, p - a) > 0) inside = !inside;
    }
    return inside ? 1 : 0;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<P> poly(n);
    for (auto& p : poly) cin >> p.x >> p.y;
    const char* names[3] = {"OUTSIDE", "INSIDE", "BOUNDARY"};
    while (m--) {
        P p;
        cin >> p.x >> p.y;
        cout << names[pointInPolygon(poly, p)] << '\n';
    }
}
```

7.3 Convex hull (monotone chain) with area

Prints the hull (counter-clockwise, collinear points dropped) and its area.

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct P {
    ll x, y;
    bool operator<(const P& o) const { return x != o.x ? x < o.x : y < o.y; }
    bool operator==(const P& o) const { return x == o.x && y == o.y; }
    P operator-(const P& o) const { return {x - o.x, y - o.y}; }
};
ll cross(P a, P b) { return a.x * b.y - a.y * b.x; }
ll cross(P o, P a, P b) { return cross(a - o, b - o); }

vector<P> convexHull(vector<P> pts) {
    sort(pts.begin(), pts.end());
    pts.erase(unique(pts.begin(), pts.end()), pts.end());
    int n = pts.size();
    if (n < 3) return pts;
    vector<P> h(2 * n);
    int k = 0;
    for (int i = 0; i < n; i++) {
        while (k >= 2 && cross(h[k - 2], h[k - 1], pts[i]) <= 0) k--;
        h[k++] = pts[i];
    }
    for (int i = n - 2, lo = k + 1; i >= 0; i--) {
        while (k >= lo && cross(h[k - 2], h[k - 1], pts[i]) <= 0) k--;
        h[k++] = pts[i];
    }
    h.resize(k - 1);
    return h;
}

ll area2(const vector<P>& poly) {                    // twice the signed area
    ll s = 0;
    for (size_t i = 0; i < poly.size(); i++) s += cross(poly[i], poly[(i + 1) %
poly.size()]);
    return s;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<P> pts(n);
    for (auto& p : pts) cin >> p.x >> p.y;
    vector<P> h = convexHull(pts);
    cout << h.size() << '\n';
    for (auto& p : h) cout << p.x << ' ' << p.y << '\n';
    ll a2 = h.size() >= 3 ? area2(h) : 0;
    cout << "area " << a2 / 2 << (a2 % 2 ? ".5" : "") << '\n';
}
```

Verified against a brute force that marks a point as a hull vertex iff it lies in no triangle or segment of other points.

7.4 CSES 2193 — Polygon Lattice Points

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    int n;
    cin >> n;
    vector<ll> x(n), y(n);
    for (int i = 0; i < n; i++) cin >> x[i] >> y[i];
    ll twiceArea = 0, boundary = 0;
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        twiceArea += x[i] * y[j] - x[j] * y[i];
        boundary += gcd(llabs(x[j] - x[i]), llabs(y[j] - y[i]));
    }
    twiceArea = llabs(twiceArea);
    cout << (twiceArea - boundary + 2) / 2 << ' ' << boundary << '\n';
}
```

7.5 Point in a convex polygon, O(log n)

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct P {
    ll x, y;
    P operator-(const P& o) const { return {x - o.x, y - o.y}; }
};
ll cross(P a, P b) { return a.x * b.y - a.y * b.x; }
ll cross(P o, P a, P b) { return cross(a - o, b - o); }

// poly: counter-clockwise, strictly convex, n >= 3. 0 out, 1 in, 2 boundary.
int inConvex(const vector<P>& poly, P p) {
    int n = poly.size();
    P o = poly[0];
    if (cross(o, poly[1], p) < 0 || cross(o, poly[n - 1], p) > 0) return 0;
    int lo = 1, hi = n - 1;
    while (hi - lo > 1) {
        int mid = (lo + hi) / 2;
        if (cross(o, poly[mid], p) >= 0) lo = mid; else hi = mid;
    }
    ll c = cross(poly[lo], poly[lo + 1], p);
    if (c < 0) return 0;
    if (c == 0) return 2;
    if ((lo == 1 && cross(o, poly[1], p) == 0) ||
```

```
        (lo + 1 == n - 1 && cross(o, poly[n - 1], p) == 0)) return 2;
    return 1;
}

int main() {
    int n, m;
    cin >> n >> m;
    vector<P> poly(n);
    for (auto& p : poly) cin >> p.x >> p.y;
    const char* names[3] = {"OUTSIDE", "INSIDE", "BOUNDARY"};
    while (m--) {
        P p;
        cin >> p.x >> p.y;
        cout << names[inConvex(poly, p)] << '\n';
    }
}
```

Verified against the O(n) ray-casting version on random convex hulls and query points.

8. Common mistakes

- **`int` coordinates in cross products** (overflow at about $4.6 \cdot 10^4$).
- **Ray casting without the half-open rule**, counting vertices twice.
- **Hull with duplicates** or unhandled all-collinear input.
- **Wrong collinear policy** for the problem's hull definition.
- **`atan2`/`acos` in decisions**, NaN from `acos(1.0000000001)`.
- **Fixed epsilon** independent of the scale of coordinates.
- **Printing `-0.000`**: add `0.0` or clamp tiny values before printing.
- **Assuming orientation**: input polygons may be clockwise; normalise with the sign of the area.

9. Practice problems

- CSES 2189 — Point Location Test: https://cses.fi/problemset/task/2189
- CSES 2190 — Line Segment Intersection: https://cses.fi/problemset/task/2190
- CSES 2191 — Polygon Area: https://cses.fi/problemset/task/2191
- CSES 2192 — Point in Polygon: https://cses.fi/problemset/task/2192
- CSES 2193 — Polygon Lattice Points: https://cses.fi/problemset/task/2193
- CSES 2195 — Convex Hull: https://cses.fi/problemset/task/2195
- Kattis `convexhull`: https://open.kattis.com/problems/convexhull
- Kattis `pointinpolygon`: https://open.kattis.com/problems/pointinpolygon
- Kattis `polygonarea`: https://open.kattis.com/problems/polygonarea
- Kattis `segmentintersection`: https://open.kattis.com/problems/segmentintersection
- Codeforces 166B — Polygons: https://codeforces.com/problemset/problem/166/B
- Codeforces 598C — Nearest vectors: https://codeforces.com/problemset/problem/598/C

- Codeforces 1C — Ancient Berland Circus: https://codeforces.com/problemset/problem/1/C

10. Further reading

- cp-algorithms: "Basic geometry", "Oriented area of a triangle", "Check if two segments intersect", "Point location in convex polygon", "Convex hull construction" (https://cp-algorithms.com).
- Victor Lecomte, *Handbook of geometry for competitive programmers* (free PDF) — the best treatment of precision and exact predicates.
- KACTL geometry section (https://github.com/kth-competitive-programming/kactl) — compact, tested code for the notebook.
- National track Lecture 8 continues with rotating calipers, half-plane intersection, and closest pair.